Permutation Generation with Position Restrictions

(Backtracking Approach)

1. Problem Statement

We are given a set of n distinct elements and a restriction list that tells us which positions certain elements are not allowed to occupy in a permutation. The task is to generate all permutations of the elements such that none of the restrictions are violated. A backtracking approach is used because it lets us build the permutation position by position and reject an invalid choice immediately, instead of generating all n! permutations first and filtering them afterward.

2. Input and Output

Input

- E[1..n] — the set of n elements that need to be permuted.
- Restrict[e] — for every element e, the set of positions (1 to n) at which e is not allowed to be placed.
- n — total number of elements.

Output

- All permutations of E of length n in which every element satisfies its position restriction. If an element has no restriction, it can go anywhere.

3. Idea / Approach

The algorithm fills the permutation array one position at a time, starting from position 1. At every position, we try each element that has not been used yet. Before placing the element, we check whether the current position is in that element's restricted list. If it is restricted, we skip this element without going into recursion — this is the pruning step, and it saves a lot of unnecessary work compared to generating a full permutation and checking it at the end. If the position is not restricted for that element, we place it, mark it as used, and recursively move to the next position. Once the recursive call returns, we undo the placement (unmark it) so that the same element can be tried in a different position in another branch. When the position counter exceeds n, it means all positions are filled correctly, so the current arrangement is printed as one valid answer.

4. Pseudocode

```
Algorithm: PermuteWithRestrictions

Input:
    E[1..n]        -> set of n elements to permute
    Restrict[1..n] -> restriction list, where Restrict[e] is the
                      set of positions element e is NOT allowed to occupy
    n              -> number of elements

Output:
```

```
    All permutations of E in which no element appears in a
    position it is restricted from.

Data Structures:
    Position[1..n] -> array to build the current permutation
    Used[1..n]     -> boolean array, Used[i] = true if E[i] already placed

------------------------------------------------------------

PROCEDURE PermuteWithRestrictions(E, Restrict, n)
    Position <- new array of size n
    Used     <- new boolean array of size n, all set to false
    CALL Backtrack(Position, Used, 1, E, Restrict, n)
END PROCEDURE

------------------------------------------------------------

PROCEDURE Backtrack(Position, Used, level, E, Restrict, n)
    IF level > n THEN
        PRINT Position[1..n]        // one valid permutation found
        RETURN
    END IF

    FOR i <- 1 TO n DO
        IF Used[i] = false THEN

            // ---- Pruning / constraint check ----
            IF level NOT IN Restrict[E[i]] THEN

                // Place element
                Position[level] <- E[i]
                Used[i] <- true

                // Recurse to next position
                CALL Backtrack(Position, Used, level + 1, E, Restrict, n)

                // Undo placement (backtrack step)
                Used[i] <- false
                Position[level] <- EMPTY
            END IF
        END IF
    END FOR
END PROCEDURE
```

5. Explanation of Key Steps

- Used[] array — keeps track of which elements have already been placed so the same element is not repeated in one permutation.
- Constraint check (pruning) — 'IF level NOT IN Restrict[E[i]]' is what stops the algorithm from even trying an invalid placement, so invalid branches are cut off before recursion happens.
- Recursive call — Backtrack(level + 1) tries to fill the next position, building the permutation gradually.

- Undo step — after returning from recursion, Used[i] is reset to false and Position[level] is cleared, which allows the loop to try the next candidate for that position. This is the actual 'backtracking' part.
- Base case — when level > n, all n positions are filled without breaking any restriction, so it is a valid permutation and gets printed.

6. Dry Run Example

Let E = {A, B, C} with Restrict(A) = {1} and Restrict(C) = {3}, meaning A cannot be first and C cannot be last.

```
E = {A, B, C}
Restrict(A) = {1}      -> A cannot be placed at position 1
Restrict(B) = { }      -> B has no restriction
Restrict(C) = {3}      -> C cannot be placed at position 3

Position 1 : try A -> rejected (1 is restricted for A)
             try B -> accepted, Position = [B, _, _]
             try C -> accepted, Position = [C, _, _]  (explored after B branch
ends)

Branch [B, _, _]:
  Position 2 : try A -> accepted, Position = [B, A, _]
      Position 3 : try C -> rejected (3 is restricted for C)
      -> dead end, backtrack
  Position 2 : try C -> accepted, Position = [B, C, _]
      Position 3 : try A -> accepted -> Permutation [B, C, A] FOUND

Branch [C, _, _]:
  Position 2 : try A -> accepted, Position = [C, A, _]
      Position 3 : try B -> accepted -> Permutation [C, A, B] FOUND
  Position 2 : try B -> accepted, Position = [C, B, _]
      Position 3 : try A -> accepted -> Permutation [C, B, A] FOUND

Final valid permutations = { [B, C, A], [C, A, B], [C, B, A] }
```

7. Why Duplicates and Invalid Permutations Do Not Occur

- The Used[] array guarantees every element is placed exactly once in each branch, so no permutation repeats an element — this avoids duplicate elements inside one arrangement.
- Because each recursive call explores a different unused element at a different position, and the array is restored after every call, no two branches can ever produce the exact same sequence — so no duplicate permutations are printed.
- The restriction check happens before recursion, so a branch that would create an invalid permutation is never even entered — meaning only valid permutations reach the print statement.

8. Time Complexity

In the worst case (no restrictions at all), the algorithm still explores all n! permutations, so the worst-case time complexity is O(n × n!), since building each permutation of length n takes O(n) work. With restrictions in place, many branches get pruned early, so in practice the algorithm runs much faster than the worst case.

9. Conclusion

This backtracking method solves the restricted permutation problem efficiently by combining recursive branching with an early pruning check. Instead of wasting time generating every permutation and checking it afterward, invalid placements are rejected immediately at the position level, which makes the algorithm both correct and efficient for this kind of constraint-based problem.